RutaAnalista

GRATIS · PRÁCTICA DE ENTREVISTA

15 ejercicios SQL que aparecen en entrevistas de Analista de Datos

Con una base de datos real de e-commerce para practicar, pistas y soluciones explicadas paso a paso.

Por **Andrés Lizarazo · BI & Analytics Manager**

Antes de empezar

Hola, soy Andrés. Llevo más de 5 años trabajando con datos y hoy soy BI & Analytics Manager con clientes en Estados Unidos. Armé estos ejercicios con el tipo de preguntas que aparecen en pruebas técnicas para Analista de Datos: no son trucos de memoria, son preguntas de negocio que se responden con SQL.

1. Abre la base de datos (5 minutos)

1. Descarga **tienda-practica.sqlite** (viene junto a este PDF).
2. Instala **DB Browser for SQLite** (gratis, sqlitebrowser.org) o usa **sqliteonline.com** sin instalar nada.
3. Abre el archivo, ve a la pestaña "Ejecutar SQL" y prueba: `SELECT * FROM pedidos LIMIT 10;`

¿Prefieres Excel, Power BI o PostgreSQL? Los mismos datos vienen en CSV.

2. Conoce los datos

Andina Market es una tienda online ficticia que vende en 6 países de Latinoamérica (2024–2025). Las cifras son inventadas, pero los patrones son realistas: temporadas, Black Friday, devoluciones y hasta datos sucios.

clientes

cliente_id, nombre, email, ciudad, pais, fecha_registro, canal_adquisicion

pedidos

pedido_id, cliente_id, fecha_pedido, estado, metodo_pago, ciudad_envio, pais_envio, costo_envio

detalle_pedido

pedido_id, producto_id, cantidad, precio_unitario, descuento (0.10 = 10%)

productos / categorias

producto_id, nombre, categoria_id, precio, costo, activo · categoria_id, nombre

Relaciones: clientes 1—N pedidos · pedidos 1—N detalle_pedido · productos 1—N detalle_pedido · categorias 1—N productos.

3. Definiciones de negocio (úsalas en todos los ejercicios)

Venta válida: pedidos con `estado IN ('entregado', 'enviado')`. Los cancelados y devueltos no cuentan como ingreso.

Ingreso de una línea: `cantidad * precio_unitario * (1 - descuento)`. No incluye el costo de envío.

Ticket promedio: ingreso total / número de pedidos.

4. Cómo sacarle provecho

- Intenta cada ejercicio **sin mirar la solución** durante al menos 10 minutos. Si te bloqueas, lee la pista.
- Compara tu resultado con la tabla de la solución. Si no coincide, busca por qué: ahí se aprende.
- Practica explicarlo en voz alta: en la entrevista importa tanto el SQL como cómo lo razonas.

Los 15 ejercicios

Primero los enunciados. Las soluciones están en la siguiente sección.

1. Clientes por país FÁCIL

¿Cuántos clientes registrados hay en cada país? Ordena de mayor a menor.

Pista: Agrupa por la columna `pais` de `clientes`.

2. Los 5 productos activos más caros FÁCIL

Lista nombre y precio de los 5 productos más caros que siguen activos (`activo = 1`).

Pista: Filtra con WHERE antes de ordenar y usa LIMIT.

3. Pedidos por estado y su porcentaje FÁCIL

¿Cuántos pedidos hay en cada estado (entregado, cancelado, devuelto, enviado) y qué porcentaje del total representa cada uno?

Pista: El total general lo puedes sacar con una subconsulta o con `SUM(COUNT(*)) OVER ()`.

4. Ingreso total de 2025 FÁCIL

Calcula el ingreso total de 2025. Usa la definición de ingreso de la página "Antes de empezar".

Pista: Une `pedidos` con `detalle_pedido`, filtra estados válidos y el año.

5. Ingreso por mes INTERMEDIO

Muestra el ingreso de cada mes de 2025 (formato AAAA-MM) y el número de pedidos.

Pista: Extrae el mes con `strftime('%Y-%m', fecha_pedido)` o `substr(fecha_pedido, 1, 7)`.

6. Top 5 productos por ingreso en 2025 INTERMEDIO

¿Cuáles fueron los 5 productos con más ingreso en 2025? Muestra también su categoría y las unidades vendidas.

Pista: Necesitas 4 tablas: pedidos, detalle_pedido, productos y categorias.

7. Clientes que nunca han comprado INTERMEDIO

¿Cuántos clientes se registraron pero nunca hicieron un pedido? Lista los 5 más antiguos.

Pista: LEFT JOIN y quédate con los que no encontraron pareja (`IS NULL`), o usa `NOT EXISTS` .

8. Ticket promedio por método de pago INTERMEDIO

¿Cuál es el valor promedio de un pedido entregado (ticket promedio) según el método de pago?

Pista: Primero calcula el total de cada pedido y después promedia. Un CTE (`WITH`) te ayuda.

9. Margen bruto por categoría INTERMEDIO

Calcula el margen bruto (%) de cada categoría: (ingreso − costo) / ingreso.

Pista: El costo de cada línea es `cantidad * costo` (de la tabla productos).

10. Clientes frecuentes INTERMEDIO

Lista los clientes con 5 o más pedidos entregados: nombre, país y número de pedidos.

Pista: Filtrar grupos se hace con HAVING, no con WHERE.

11. Crecimiento mes a mes AVANZADO

Para cada mes, muestra el ingreso, el del mes anterior y la variación porcentual.

Pista: La función de ventana `LAG()` trae el valor de la fila anterior.

12. Top 3 productos dentro de cada categoría AVANZADO

Muestra los 3 productos con más ingreso de cada categoría.

Pista: `ROW_NUMBER() OVER (PARTITION BY categoria ORDER BY ingreso DESC)` y luego filtra posición ≤ 3.

13. Emails duplicados (calidad de datos) AVANZADO

Algunos clientes se registraron dos veces con el mismo email escrito distinto (mayúsculas o espacios). Encuéntralos.

Pista: Normaliza con `LOWER(TRIM(email))` y agrupa.

14. Días hasta la segunda compra AVANZADO

Para los clientes con al menos 2 pedidos entregados, ¿cuántos días pasan en promedio entre la primera y la segunda compra?

Pista: Numera los pedidos de cada cliente con `ROW_NUMBER()` o usa `LEAD()` sobre la fecha.

15. Regla de Pareto (80/20) AVANZADO

¿Cuántos productos generan el 80% del ingreso total? Muestra el ranking con el % acumulado.

Pista: Ordena productos por ingreso y calcula una suma acumulada con `SUM(...) OVER (ORDER BY ...)`.

Soluciones explicadas

1. Clientes por país FÁCIL

¿Cuántos clientes registrados hay en cada país? Ordena de mayor a menor.

Qué evalúa el entrevistador: Lo básico de **GROUP BY** y **ORDER BY**. Lo piden para entrar en calor.

```sql
SELECT pais, COUNT(*) AS clientes
FROM clientes
GROUP BY pais
ORDER BY clientes DESC;
```

Resultado

pais	clientes
Colombia	1,679
México	1,062
Perú	487
Chile	385
Ecuador	339
Argentina	266

- `COUNT(*)` cuenta filas dentro de cada grupo.
- Puedes ordenar por el alias `clientes` que definiste en el SELECT.

2. Los 5 productos activos más caros FÁCIL

Lista nombre y precio de los 5 productos más caros que siguen activos (`activo = 1`).

Qué evalúa el entrevistador: Orden lógico de una consulta: primero se filtra (WHERE), luego se ordena y al final se limita.

```sql
SELECT nombre, precio
FROM productos
WHERE activo = 1
ORDER BY precio DESC
LIMIT 5;
```

Resultado

nombre	precio
Smartwatch deportivo	119.00
Disco SSD 1TB	89.00
Chaqueta impermeable	84.00
Teclado mecánico	79.00
Tenis urbanos	69.90

- En SQL Server se usa `SELECT TOP 5`; en SQLite, PostgreSQL, MySQL y BigQuery se usa `LIMIT`.

3. Pedidos por estado y su porcentaje FÁCIL

¿Cuántos pedidos hay en cada estado (entregado, cancelado, devuelto, enviado) y qué porcentaje del total representa cada uno?

Qué evalúa el entrevistador: Saber calcular participaciones (%). Ojo con la división entera: `100 * 3 / 4` da 75, pero `3 / 4` da 0 en muchos motores.

```sql
SELECT estado,
       COUNT(*) AS pedidos,
       ROUND(100.0 * COUNT(*) / (SELECT COUNT(*) FROM pedidos), 1) AS pct
FROM pedidos
GROUP BY estado
ORDER BY pedidos DESC;
```

Resultado

estado	pedidos	pct
entregado	7,456	87.40
cancelado	519	6.10
devuelto	477	5.60
enviado	76	0.90

- Multiplicar por `100.0` (con decimal) obliga a una división con decimales.
- La tasa de cancelación y de devolución son métricas que cualquier e-commerce mira cada semana.

4. Ingreso total de 2025 FÁCIL

Calcula el ingreso total de 2025. Usa la definición de ingreso de la página "Antes de empezar".

Qué evalúa el entrevistador: Que conectes tablas y, sobre todo, que **preguntes la definición de la métrica**. ¿Incluye cancelados? ¿Descuentos? ¿Envío? Preguntarlo suma puntos.

```sql
SELECT ROUND(SUM(d.cantidad * d.precio_unitario * (1 - d.descuento)), 2) AS ingreso_2025
FROM pedidos p
JOIN detalle_pedido d ON d.pedido_id = p.pedido_id
WHERE p.estado IN ('entregado', 'enviado')
  AND p.fecha_pedido >= '2025-01-01' AND p.fecha_pedido < '2026-01-01';
```

Resultado

ingreso_2025
313,958.15

- Las fechas están guardadas como texto ISO (AAAA-MM-DD), así que se pueden comparar como texto.
- Filtrar con un rango (`>= y <`) es mejor práctica que aplicar funciones a la columna: aprovecha índices.

5. Ingreso por mes INTERMEDIO

Muestra el ingreso de cada mes de 2025 (formato AAAA-MM) y el número de pedidos.

Qué evalúa el entrevistador: La consulta más común del trabajo real: una serie de tiempo. Fíjate en contar pedidos con `COUNT(DISTINCT ...)`.

```sql
SELECT strftime('%Y-%m', p.fecha_pedido) AS mes,
       COUNT(DISTINCT p.pedido_id) AS pedidos,
       ROUND(SUM(d.cantidad * d.precio_unitario * (1 - d.descuento)), 2) AS ingreso
FROM pedidos p
JOIN detalle_pedido d ON d.pedido_id = p.pedido_id
WHERE p.estado IN ('entregado', 'enviado')
  AND p.fecha_pedido >= '2025-01-01'
GROUP BY mes
ORDER BY mes;
```

Resultado

mes	pedidos	ingreso
2025-01	275	17,983.56
2025-02	295	18,401.70
2025-03	361	21,728.28
2025-04	350	24,120.36
2025-05	406	26,255.41
2025-06	349	23,434.11

... 6 filas más

- Al unir con el detalle, cada pedido aparece una vez por producto. Por eso se cuenta con `COUNT(DISTINCT pedido_id)`; `COUNT(*)` contaría líneas, no pedidos.
- En PostgreSQL sería `DATE_TRUNC('month', fecha)`; en BigQuery, `DATE_TRUNC(fecha, MONTH)`.

6. Top 5 productos por ingreso en 2025 INTERMEDIO

¿Cuáles fueron los 5 productos con más ingreso en 2025? Muestra también su categoría y las unidades vendidas.

Qué evalúa el entrevistador: JOINs de varias tablas sin duplicar filas, y elegir bien la clave de agrupación.

```sql
SELECT pr.nombre AS producto,
       c.nombre AS categoria,
       SUM(d.cantidad) AS unidades,
       ROUND(SUM(d.cantidad * d.precio_unitario * (1 - d.descuento)), 2) AS ingreso
FROM pedidos p
JOIN detalle_pedido d ON d.pedido_id = p.pedido_id
JOIN productos pr     ON pr.producto_id = d.producto_id
JOIN categorias c     ON c.categoria_id = pr.categoria_id
WHERE p.estado IN ('entregado', 'enviado')
  AND p.fecha_pedido >= '2025-01-01'
GROUP BY pr.producto_id, pr.nombre, c.nombre
ORDER BY ingreso DESC
LIMIT 5;
```

Resultado

producto	categoria	unidades	ingreso
Morral deportivo	Deportes	828	33,555.68
Lego clásico	Juguetes	622	31,815.85
Medias x6	Moda	1,418	19,018.39
Licuadora	Hogar	335	18,849.13
Audífonos inalámbricos	Tecnología	184	11,418.68

- Agrupa por el id del producto (no solo por el nombre): dos productos distintos podrían llamarse igual.

7. Clientes que nunca han comprado INTERMEDIO

¿Cuántos clientes se registraron pero nunca hicieron un pedido? Lista los 5 más antiguos.

Qué evalúa el entrevistador: Una pregunta clásica para ver si entiendes la diferencia entre INNER JOIN y LEFT JOIN.

```sql
SELECT c.cliente_id, c.nombre, c.fecha_registro, c.canal_adquisicion
FROM clientes c
LEFT JOIN pedidos p ON p.cliente_id = c.cliente_id
WHERE p.pedido_id IS NULL
ORDER BY c.fecha_registro
LIMIT 5;

-- Para el total: SELECT COUNT(*) FROM clientes c
--   WHERE NOT EXISTS (SELECT 1 FROM pedidos p WHERE p.cliente_id = c.cliente_id);
```

Resultado

cliente_id	nombre	fecha_registro	canal_adquisicion
1,367	Carlos Cruz	2024-01-03	TikTok
1,690	Alejandro Rojas	2024-01-03	Email
3,971	María Hernández	2024-01-04	Referido
2,183	Fabián Álvarez	2024-01-07	TikTok
950	Tomás Silva	2024-01-08	Google Ads

- El LEFT JOIN conserva todos los clientes; los que no tienen pedidos quedan con las columnas de `pedidos` en NULL.
- Negocio: estos clientes son una audiencia ideal para un email de “primera compra”.

8. Ticket promedio por método de pago INTERMEDIO

¿Cuál es el valor promedio de un pedido entregado (ticket promedio) según el método de pago?

Qué evalúa el entrevistador: La trampa: si promedias directamente las líneas del detalle obtienes el promedio por **producto**, no por **pedido**.

```sql
WITH totales AS (
  SELECT p.pedido_id, p.metodo_pago,
         SUM(d.cantidad * d.precio_unitario * (1 - d.descuento)) AS total
  FROM pedidos p
  JOIN detalle_pedido d ON d.pedido_id = p.pedido_id
  WHERE p.estado = 'entregado'
  GROUP BY p.pedido_id, p.metodo_pago
)
SELECT metodo_pago,
       COUNT(*) AS pedidos,
       ROUND(AVG(total), 2) AS ticket_promedio
FROM totales
GROUP BY metodo_pago
ORDER BY ticket_promedio DESC;
```

Resultado

metodo_pago	pedidos	ticket_promedio
transferencia	599	67.83
billetera_digital	834	67.56
efectivo	735	66.55
pse	734	64.75
tarjeta_credito	2,922	64.03
tarjeta_debito	1,282	63.65

... 1 filas más

- El CTE crea una “tabla temporal” con una fila por pedido. Sobre ella el promedio sí es por pedido.
- En entrevistas, explicar en voz alta este paso de “grano” (nivel de detalle) impresiona.

9. Margen bruto por categoría INTERMEDIO

Calcula el margen bruto (%) de cada categoría: (ingreso − costo) / ingreso.

Qué evalúa el entrevistador: Pensar en rentabilidad, no solo en ventas. Una categoría que vende mucho puede dejar poco.

```sql
SELECT c.nombre AS categoria,
       ROUND(SUM(d.cantidad * d.precio_unitario * (1 - d.descuento)), 0) AS ingreso,
       ROUND(100.0 * SUM(d.cantidad * (d.precio_unitario * (1 - d.descuento) - pr.costo))
             / SUM(d.cantidad * d.precio_unitario * (1 - d.descuento)), 1) AS margen_pct
FROM pedidos p
JOIN detalle_pedido d ON d.pedido_id = p.pedido_id
JOIN productos pr     ON pr.producto_id = d.producto_id
JOIN categorias c     ON c.categoria_id = pr.categoria_id
WHERE p.estado IN ('entregado', 'enviado')
GROUP BY c.nombre
ORDER BY margen_pct DESC;
```

Resultado

categoria	ingreso	margen_pct
Belleza	49,299.00	57.90
Moda	67,482.00	52.20
Hogar	63,128.00	42.50
Juguetes	71,262.00	42.20
Mascotas	30,018.00	37.00
Deportes	97,178.00	36.80

… 2 filas más

- Un porcentaje de un grupo se calcula como **suma / suma**, nunca como promedio de porcentajes.
- Hallazgo: Tecnología vende bien pero tiene el margen más bajo.

10. Clientes frecuentes INTERMEDIO

Lista los clientes con 5 o más pedidos entregados: nombre, país y número de pedidos.

Qué evalúa el entrevistador: La diferencia entre WHERE (filtra filas antes de agrupar) y HAVING (filtra grupos después).

```sql
SELECT c.nombre, c.pais, COUNT(*) AS pedidos
FROM pedidos p
JOIN clientes c ON c.cliente_id = p.cliente_id
WHERE p.estado = 'entregado'
GROUP BY c.cliente_id, c.nombre, c.pais
HAVING COUNT(*) >= 5
ORDER BY pedidos DESC, c.nombre;
```

Resultado

nombre	pais	pedidos
Diego León	Colombia	10
Rodrigo Lizarazo	Colombia	10
Diana Castro	Colombia	9
Valentina Lizarazo	México	9
Carlos Gómez	Colombia	8
Emilio Vega	México	8

... 257 filas más

- WHERE filtra los pedidos entregados; HAVING se queda con los clientes que cumplen la condición.

11. Crecimiento mes a mes AVANZADO

Para cada mes, muestra el ingreso, el del mes anterior y la variación porcentual.

Qué evalúa el entrevistador: Funciones de ventana: el tema que más separa a un perfil junior de uno listo para trabajar.

```sql
WITH mensual AS (
  SELECT strftime('%Y-%m', p.fecha_pedido) AS mes,
         SUM(d.cantidad * d.precio_unitario * (1 - d.descuento)) AS ingreso
  FROM pedidos p
  JOIN detalle_pedido d ON d.pedido_id = p.pedido_id
  WHERE p.estado IN ('entregado', 'enviado')
  GROUP BY mes
)
SELECT mes,
       ROUND(ingreso, 0) AS ingreso,
       ROUND(LAG(ingreso) OVER (ORDER BY mes), 0) AS ingreso_mes_anterior,
       ROUND(100.0 * (ingreso - LAG(ingreso) OVER (ORDER BY mes))
             / LAG(ingreso) OVER (ORDER BY mes), 1) AS variacion_pct
FROM mensual
ORDER BY mes;
```

Resultado

mes	ingreso	ingreso_mes_anterior	variacion_pct
2024-01	3,546.00	NULL	NULL
2024-02	5,608.00	3,546.00	58.20
2024-03	8,568.00	5,608.00	52.80
2024-04	7,977.00	8,568.00	-6.90
2024-05	15,477.00	7,977.00	94.00
2024-06	12,247.00	15,477.00	-20.90

... 18 filas más

- El primer mes queda en NULL porque no tiene mes anterior: es correcto, no un error.
- Para comparar con el mismo mes del año anterior usarías `LAG(ingreso, 12)`.

12. Top 3 productos dentro de cada categoría AVANZADO

Muestra los 3 productos con más ingreso de cada categoría.

Qué evalúa el entrevistador: El famoso “top N por grupo”. Aparece en casi todas las pruebas técnicas.

```sql
WITH ventas AS (
  SELECT c.nombre AS categoria, pr.nombre AS producto,
         SUM(d.cantidad * d.precio_unitario * (1 - d.descuento)) AS ingreso
  FROM pedidos p
  JOIN detalle_pedido d ON d.pedido_id = p.pedido_id
  JOIN productos pr     ON pr.producto_id = d.producto_id
  JOIN categorias c     ON c.categoria_id = pr.categoria_id
  WHERE p.estado IN ('entregado', 'enviado')
  GROUP BY c.nombre, pr.producto_id, pr.nombre
), ranking AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY categoria ORDER BY ingreso DESC) AS puesto
  FROM ventas
)
SELECT categoria, puesto, producto, ROUND(ingreso, 0) AS ingreso
FROM ranking
WHERE puesto <= 3
ORDER BY categoria, puesto;
```

Resultado

categoria	puesto	producto	ingreso
Belleza	1	Plancha de pelo	16,740.00
Belleza	2	Crema hidratante	7,298.00
Belleza	3	Perfume 50ml	6,699.00
Deportes	1	Morral deportivo	53,114.00
Deportes	2	Botella térmica	16,471.00
Deportes	3	Mat de yoga	13,805.00

… 18 filas más

- No puedes filtrar una función de ventana en el mismo WHERE; por eso se calcula en un CTE y se filtra afuera.
- Si hay empates y los quieres incluir, usa `DENSE_RANK()` en vez de `ROW_NUMBER()`.

13. Emails duplicados (calidad de datos) AVANZADO

Algunos clientes se registraron dos veces con el mismo email escrito distinto (mayúsculas o espacios). Encuéntralos.

Qué evalúa el entrevistador: Criterio de calidad de datos: en el trabajo real los datos nunca vienen limpios, y quien lo detecta se gana la confianza del equipo.

```sql
SELECT LOWER(TRIM(email)) AS email_normalizado,
       COUNT(*) AS registros,
       GROUP_CONCAT(cliente_id, ', ') AS ids
FROM clientes
GROUP BY LOWER(TRIM(email))
HAVING COUNT(*) > 1
ORDER BY registros DESC, email_normalizado;
```

Resultado

email_normalizado	registros	ids
alejandra.quintero646@gmail.com	2	1958, 4203
alejandra.salazar668@outlook.com	2	3425, 4218
alejandro.duarte924@gmail.com	2	634, 4205
carolina.lizarazo124@hotmail.com	2	1980, 4213
catalina.sanchez949@yahoo.com	2	2608, 4214
david.mendoza772@yahoo.com	2	1821, 4216

… 14 filas más

- Sin limpiar, contarías clientes de más y las tasas de recompra saldrían mal.
- `GROUP_CONCAT` (SQLite/MySQL) equivale a `STRING_AGG` en PostgreSQL, BigQuery y SQL Server.

14. Días hasta la segunda compra AVANZADO

Para los clientes con al menos 2 pedidos entregados, ¿cuántos días pasan en promedio entre la primera y la segunda compra?

Qué evalúa el entrevistador: Pensar en el comportamiento del cliente. Este dato define cuándo enviar el email de recompra.

```sql
WITH ordenados AS (
  SELECT cliente_id, fecha_pedido,
         LEAD(fecha_pedido) OVER (PARTITION BY cliente_id ORDER BY fecha_pedido,
pedido_id) AS siguiente,
         ROW_NUMBER()       OVER (PARTITION BY cliente_id ORDER BY fecha_pedido,
pedido_id) AS n
  FROM pedidos
  WHERE estado = 'entregado'
)
SELECT COUNT(*) AS clientes_con_2_compras,
       ROUND(AVG(julianday(siguiente) - julianday(fecha_pedido)), 1) AS dias_promedio,
       MIN(julianday(siguiente) - julianday(fecha_pedido)) AS minimo,
       MAX(julianday(siguiente) - julianday(fecha_pedido)) AS maximo
FROM ordenados
WHERE n = 1 AND siguiente IS NOT NULL;
```

Resultado

clientes_con_2_compras	dias_promedio	minimo	maximo
1,973	64.00	0.00	679.00

- `julianday()` convierte la fecha a número de días en SQLite. En PostgreSQL basta con restar fechas; en BigQuery, `DATE_DIFF`.
- Agrega `pedido_id` al ORDER BY para desempatar pedidos del mismo día.

15. Regla de Pareto (80/20) AVANZADO

¿Cuántos productos generan el 80% del ingreso total? Muestra el ranking con el % acumulado.

Qué evalúa el entrevistador: Combinar ventanas con totales. Una respuesta de negocio clara: "X de 64 productos hacen el 80%".

```sql
WITH ingreso_producto AS (
  SELECT pr.nombre AS producto,
         SUM(d.cantidad * d.precio_unitario * (1 - d.descuento)) AS ingreso
  FROM pedidos p
  JOIN detalle_pedido d ON d.pedido_id = p.pedido_id
  JOIN productos pr     ON pr.producto_id = d.producto_id
  WHERE p.estado IN ('entregado', 'enviado')
  GROUP BY pr.producto_id, pr.nombre
), acumulado AS (
  SELECT producto, ingreso,
         ROW_NUMBER() OVER (ORDER BY ingreso DESC) AS puesto,
         100.0 * SUM(ingreso) OVER (ORDER BY ingreso DESC ROWS BETWEEN UNBOUNDED PRECEDING
AND CURRENT ROW)
               / SUM(ingreso) OVER () AS pct_acumulado
  FROM ingreso_producto
)
SELECT puesto, producto, ROUND(ingreso, 0) AS ingreso, ROUND(pct_acumulado, 1) AS
pct_acumulado
FROM acumulado
WHERE pct_acumulado - 100.0 * ingreso / (SELECT SUM(ingreso) FROM ingreso_producto) < 80
ORDER BY puesto;
```

Resultado

puesto	producto	ingreso	pct_acumulado
1	Morral deportivo	53,114.00	10.90
2	Lego clásico	51,073.00	21.30
3	Licuadora	30,899.00	27.60
4	Medias x6	29,226.00	33.60
5	Audífonos inalámbricos	17,636.00	37.20
6	Plancha de pelo	16,740.00	40.60

... 24 filas más

- La condición final se queda con los productos que hacía falta sumar para llegar al 80%.
- Muchas empresas usan este análisis para decidir qué inventario priorizar.

¿Y ahora qué?

Si resolviste estos 15 ejercicios, ya tienes una base de SQL mejor que la de muchos candidatos. Pero el SQL solo no te consigue el trabajo. Lo que hace que te llamen es poder **mostrar** que resuelves problemas de negocio:

- Un portafolio con proyectos reales (no el típico Titanic).
- Un CV que pase los filtros automáticos (ATS) y hable de resultados.
- Preparación para la entrevista, incluida la parte en inglés si apuntas a empresas de EE.UU.

Kit Ruta Analista · $19

- Ruta de 8 semanas: qué estudiar, qué ignorar y cuándo postular.
- 60 ejercicios SQL resueltos (estos 15 son una muestra) con la base completa: ventas, inventario y marketing.
- 3 proyectos de portafolio guiados con dashboard en Excel y plantilla de README para GitHub.
- CV en español e inglés, guía de LinkedIn y 100 preguntas de entrevista con respuestas.

Pago único · Garantía de 7 días · Acceso inmediato

rutaanalista.com →

© Andrés Lizarazo. Puedes compartir este PDF completo y gratis con quien quieras. Los datos de Andina Market son ficticios.